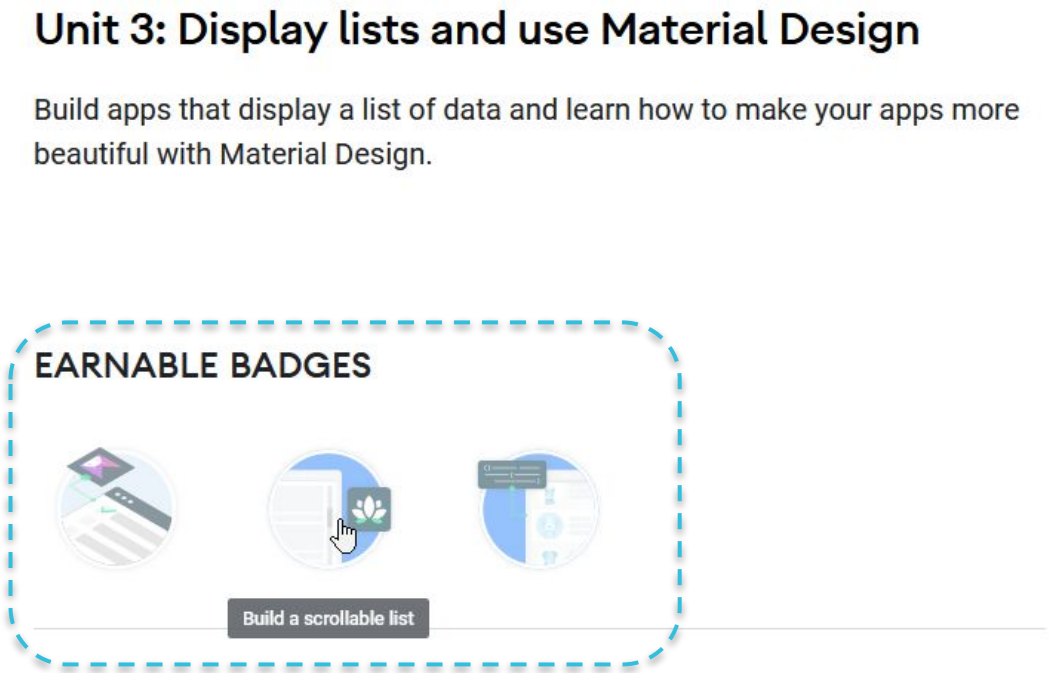
Android - Unit 3

Display lists and use Material Design

User Interfaces 2

Unit 3: Display lists and use Material Design

1. More Kotlin fundamentals
2. Build a scrollable list
3. Build beautiful apps



PW1 More Kotlin fundamentals



- Zie ook slides Kotlin Lesson 1, Kotlin Basics: van Lists and Arrays tot einde
- `List<T>,Array<T>,Set<T>,Map<K,T>...`
  - `val theList = listOf("John","Pete","George")`
    - werkt zoals Java `List.of()`
    - type inferentie: `theList` is een `List<String>`
  - `mutableListOf()...`
  - `Array<String>`
    - geen String[ ] type of {element1, element2…} initialisatie (anders dan Java)
    - `List` is gebruikelijker dan `Array`
- Omzetten naar String: `myList.contentToString()`
- + operator (Kotlin heeft operator overloading!)
- Null Safety

PW1 More Kotlin fundamentals



- Zie ook slides Kotlin Lesson 2, Functions: List filters
- list evaluation is eager (zoals in JavaScript)
- `asSequence()`, geeft lazy list evaluation (zoals `stream()` in java)
- vloeiende programeerstiijl met collection functies
  - `it`
  - `forEach()`, `map()`, `filter()`, `groupBy()`, `sortedBy()`, `fold()`, `reduce()...`

PW1 More Kotlin fundamentals



Generics, objects and extensions

- Zie ook Slides Kotlin Lesson 3: Classes and Objects, vanaf *extension functions* tot einde
- Special classes
  - enum
  - data class is als record in Java (16), maar kan ook mutable zijn
  - (singleton) objects
    - Companion objects zijn een alternatief voor een aantal toepassingen van static in java
- Pair, Triple
  - Pair: zoals Map.Entry in Java
  - enkel in Kotlin slides, komen in latere codelab aan bod
- Extension properties & extension functions

Scope functions



- chained function on an object in which the object can be accessed without reusing the variable name.

Overview of scope functions for an object o		
in function, refer to **o** using	Returns **o** (allowing chaining on o)	returns result of lambda (implicitly last statement)
`it`	`o.also{//… it …}`	**`o.let{//… it …}`**

`this`	**`o.apply{//…}`**	`o.run{//…}`
		`// met object parameter`
		`with(o){//…}`

PW1 More Kotlin fundamentals



(link)

Oefeningen:

- Practice: Classes and Collections
- Canvas: oefening 3.1-3.3



Te behalen Badge Pathway 1:
https://developer.android.com/courses/pathways/android-basics-compose-unit-3-pathway-1

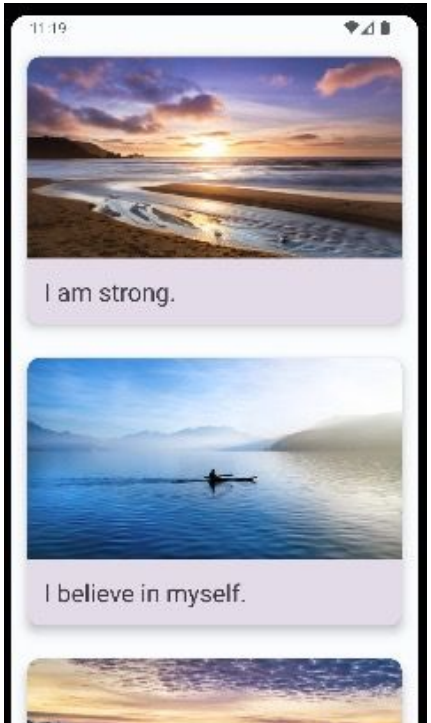
PW2 Build a scrollable list (link)



- Code Along: Add a scrollable list
- Scrolling and **lazy** composables
  - LazyColumn
  - LazyVerticalGrid & LazyHorizontalGrid

(een of meer) `LazyListScope` functies zoals `items`, met de inhoud voor de `LazyColumn`: https://developer.android.com/develop/ui/compose/lists#lazy

```
LazyColumn(modifier = modifier) {
    items(aListOfCards) {
        ListCard(
            content = it,
            modifier = Modifier.padding(8.dp)
        )
    }
}
```



PW2 Build a scrollable list (link)



Oefening: “Build a grid”

- use a LazyVerticalGrid

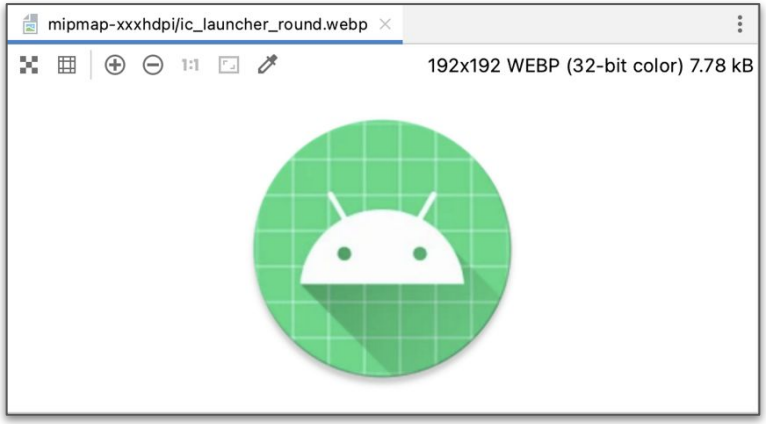


Te behalen Pathway 2:
https://developer.android.com/courses/pathways/android-basics-compose-unit-3-pathway-2

PW2 Build a scrollable list (link)



- Application icons
  - Code Along: Change the app icon
    - continue from the scrollable list codelab
  - extra uitleg om app icons te maken op basis van iconen, images of tekst



PW2 Build a scrollable list (link)



- Application icons Summary
  - Place app icon files in the `mipmap` resource directories.
  - Provide different versions of an app icon bitmap image in each density bucket (`mdpi`, `hdpi`, `xhdpi`, `xxhdpi`, `xxxhdpi`) for backwards compatibility
  - Add resource qualifiers onto resource directories to specify resources that should be used on devices with a certain configuration (`v24` or `v26`).
  - Vector drawables are Android's implementation of vector graphics. They are defined in XML as a set of points, lines, and curves, along with associated color information. Vector drawables can be scaled for any density without loss of quality.
  - Adaptive icons were introduced to the Android platform in API 26. They are made up of a foreground and background layer
  - Use **Image Asset Studio** in Android Studio to create legacy and adaptive icons for your app. Om te openen
    - klik + in resource manager
    - of rechts klik op res/drawable map + New Image/Vector Asset
  - andere icon creator: icon kitchen

PW3 Build beautiful apps (link)



- Material theming with Jetpack Compose
  - Add a theme `@Composable` at the top of your Composable tree
    - Color: generate Theme from Material Theme Builder, light/dark
    - Shape
    - Fonts (`Type.kt`)
    - Dimensions (padding...)
  - Top bar
  - Status bar
- Simple animation with Jetpack Compose

Verdieping

Testing for Accessibility: bekijk even zodat je je badge kan halen, maar behoort verder niet tot de stof

PW3 Build beautiful apps (link)



- Material design
  - Color
  - Shape
  - Text
  - Top bar
  - Status bar
- Animations

Verdieping

Testing for Accessibility: bekijk even zodat je je badge kan halen, maar behoort verder niet tot de stof

PW3 Build beautiful apps (link)



Oefeningen:

- Practice: Build Superheroes app
- Project: Create a 30 Days App



Te behalen Pathway 3:
https://developer.android.com/courses/pathways/android-basics-compose-unit-3-pathway-3

Some Tips

- Geef steeds een **`modifier`** parameter door aan elke composable met default value **`Modifier`**

Default value

```
fun GreetingImage(message: String, modifier: Modifier = Modifier)
```

- **`Image()`** composable
  - verplicht een `contentDescription` mee te geven (de alt text in android)
- Wil je een ander keyboard (bv nummers)

Nummer-keyboard

```
keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Number)
```

https://developer.android.com/reference/kotlin/androidx/compose/ui/text/input/KeyboardType

- Gebruik “Translation Editor” in Android Studio om `strings.xml` te vertalen
  - Haal op via `LocalContext.current.getString()`

